Acknowledgement

I would like to express my sincere gratitude to my respected Programming Fundamentals instructor for providing guidance, motivation and valuable knowledge throughout the development of this project. The concepts taught during the course helped me understand the practical implementation of programming techniques and problem solving skills.

I am also thankful to my institution for providing the learning environment and resources required for completing this project successfully. Special thanks to my classmates and friends who shared their ideas and provided support whenever needed.

Furthermore, I would like to acknowledge the contribution of various online resources, tutorials and Python documentation that assisted me in understanding Object-Oriented Programming (OOP) concepts and Tkinter GUI development. Their guidance played an important role in overcoming challenges faced during the implementation phase.

Finally, I am grateful for the opportunity to work on this project as it enhanced my programming skills, strengthened my understanding of software development principles and provided valuable practical experience in designing and developing a real-world application.

Abstract

The Digital Library Management System is a Python based application developed to automate basic library operations through a graphical user interface. The system allows users to add, search, borrow, return, view, and delete books efficiently. It is developed using Object Oriented Programming (OOP) concepts and Tkinter GUI, making the application easy to use and manage. The project reduces manual record keeping efforts and improves the accuracy of library management. This project also demonstrates the practical implementation of Programming Fundamentals concepts such as classes, objects, functions, loops, conditional statements, and graphical user interface development. Overall, the system provides a simple, efficient, and user-friendly solution for managing library records digitally.

Table of Contents

Introduction:

Libraries play an important role in educational institutions by providing access to books and learning resources. Managing library records manually can be time consuming and prone to errors.

The Digital Library Management System was developed to automate basic library operations. The system provides an easy way to manage books and perform common tasks such as adding new books, searching books, borrowing books, returning books, and deleting books.

The project demonstrates the practical implementation of Programming Fundamentals concepts including classes, objects, loops, conditions, functions, lists, and GUI development.

Theory:

Python Programming Language:

Python is a high-level, interpreted programming language known for its simple syntax and readability. It is widely used for software development, automation, web applications, artificial intelligence, and data analysis.

Object-Oriented Programming (OOP):

Object-Oriented Programming is a programming paradigm that organizes code into objects and classes.

Class:

A class acts as a blueprint for creating objects.

Example:

Book Class

Library Class

Object:

An object is an instance of a class.

Example:

book1 = Book("Python", "Ali", "B101", 5)

Constructor:

The constructor initializes object attributes when an object is created.

Example:

init()

Methods:

Methods are functions defined inside a class that perform specific tasks.

Examples:

add_book()

borrow_book()

return_book()

delete_book()

Tkinter GUI:

Tkinter is Python's built in GUI library. It allows developers to create windows, buttons, labels, text fields, message boxes, and tables.

The project uses Tkinter to create a user friendly graphical interface for the Digital Library Management System.

Software Used:

Visual Studio Code (VS Code):

Visual Studio Code was used as the development environment for writing, editing, and debugging Python code.

Features of VS Code:

- Syntax highlighting
- Auto-completion
- Integrated terminal
- Python extension support
- Debugging tools
- Project management

Python:

Python was used as the primary programming language for implementing all project functionalities.

Tkinter:

Tkinter was used to create the graphical user interface.

Methodology:

The development of the project followed the following steps:

1. Create the Book class to store book information.
2. Create the Library class to manage library operations
3. Store books using a Python list.
4. Implement book management functions

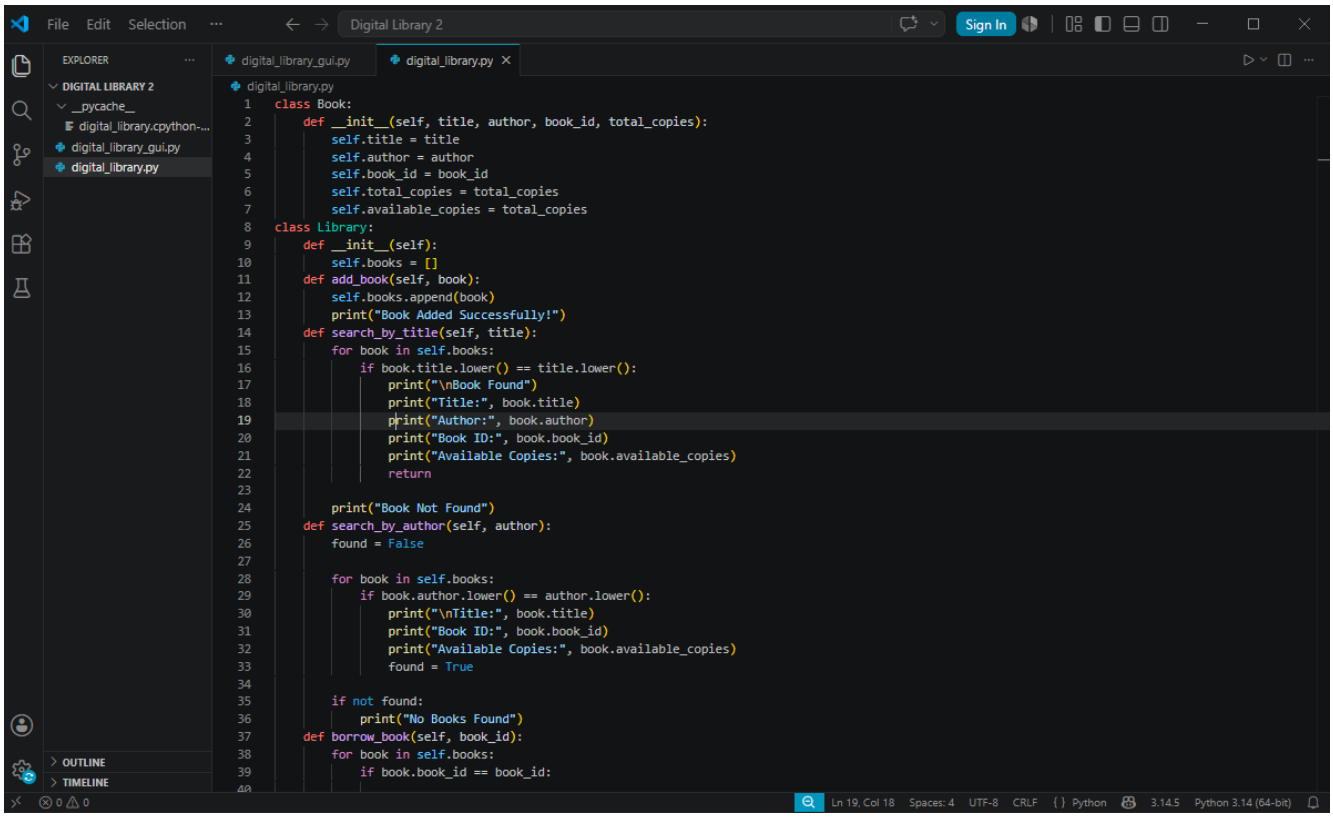
- Add Book
- Search Book by Title
- Search Book by Author
- Borrow Book
- Return Book
- Delete Book
- View Books

5. Design the GUI using Tkinter.
6. Create input fields for entering book details.
7. Create buttons for different operations.
8. Display books using a Tree view table.
9. Test all functionalities.
10. Verify output and eliminate errors.

Source Code Description:

The project consists of two main files.

File 1: digital_library.py:




```python
class Book:
    def __init__(self, title, author, book_id, total_copies):
        self.title = title
        self.author = author
        self.book_id = book_id
        self.total_copies = total_copies
        self.available_copies = total_copies
class Library:
    def __init__(self):
        self.books = []
    def add_book(self, book):
        self.books.append(book)
        print("Book Added Successfully!")
    def search_by_title(self, title):
        for book in self.books:
            if book.title.lower() == title.lower():
                print("\nBook Found")
                print("Title:", book.title)
                print("Author:", book.author)
                print("Book ID:", book.book_id)
                print("Available Copies:", book.available_copies)
                return

        print("Book Not Found")
    def search_by_author(self, author):
        found = False

        for book in self.books:
            if book.author.lower() == author.lower():
                print("\nTitle:", book.title)
                print("Book ID:", book.book_id)
                print("Available Copies:", book.available_copies)
                found = True

        if not found:
            print("No Books Found")
    def borrow_book(self, book_id):
        for book in self.books:
            if book.book_id == book_id:
```

This file contains:
Book Class:

Stores:

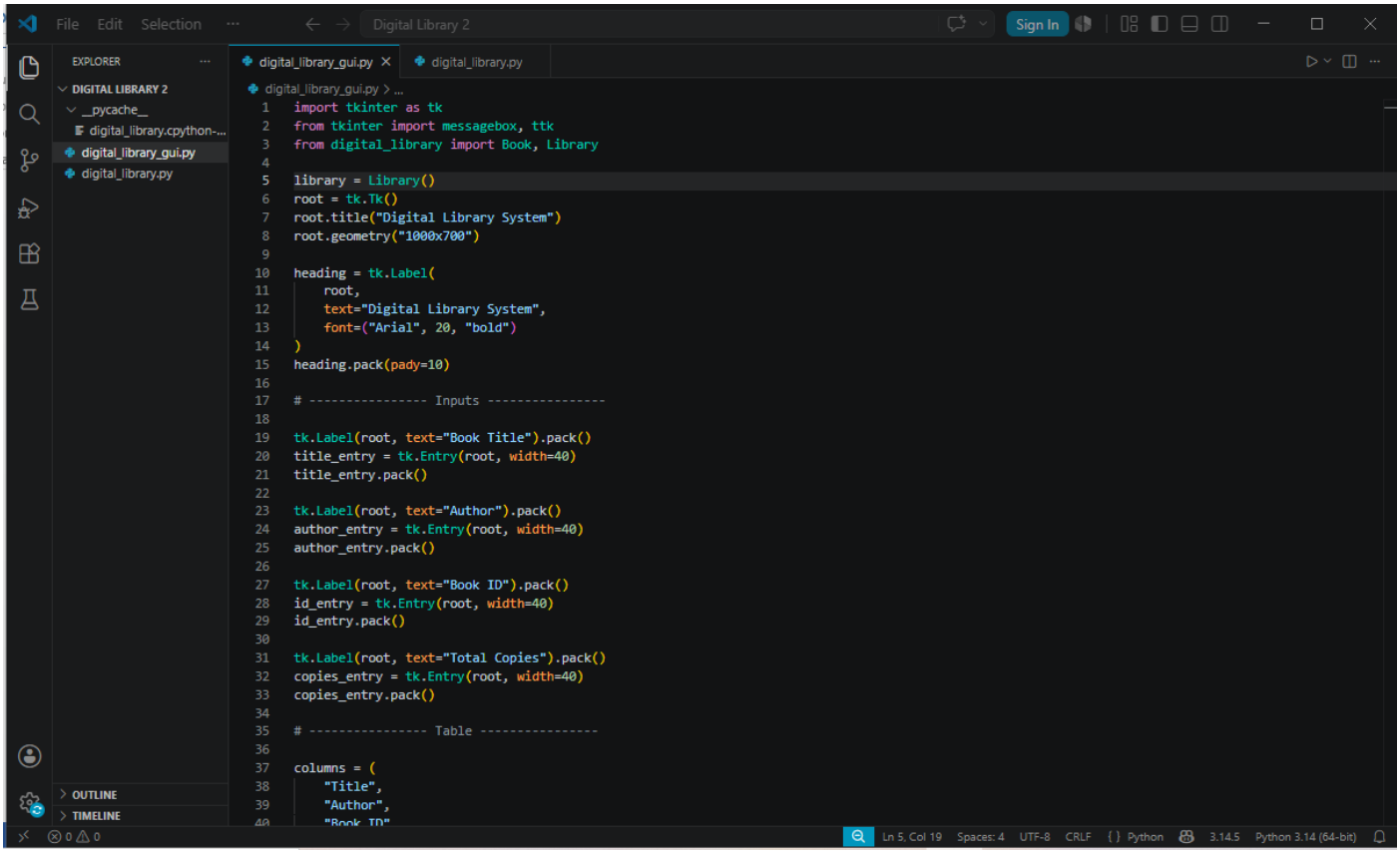
- Title
- Author
- Book ID
- Total Copies
- Available Copies

Library Class:

Provides the following methods:

- add_book()
- search_by_title()
- search_by_author()
- borrow_book()
- return_book()
- view_books()
- delete_book()

File 2: digital_library_gui.py:



This file contains:

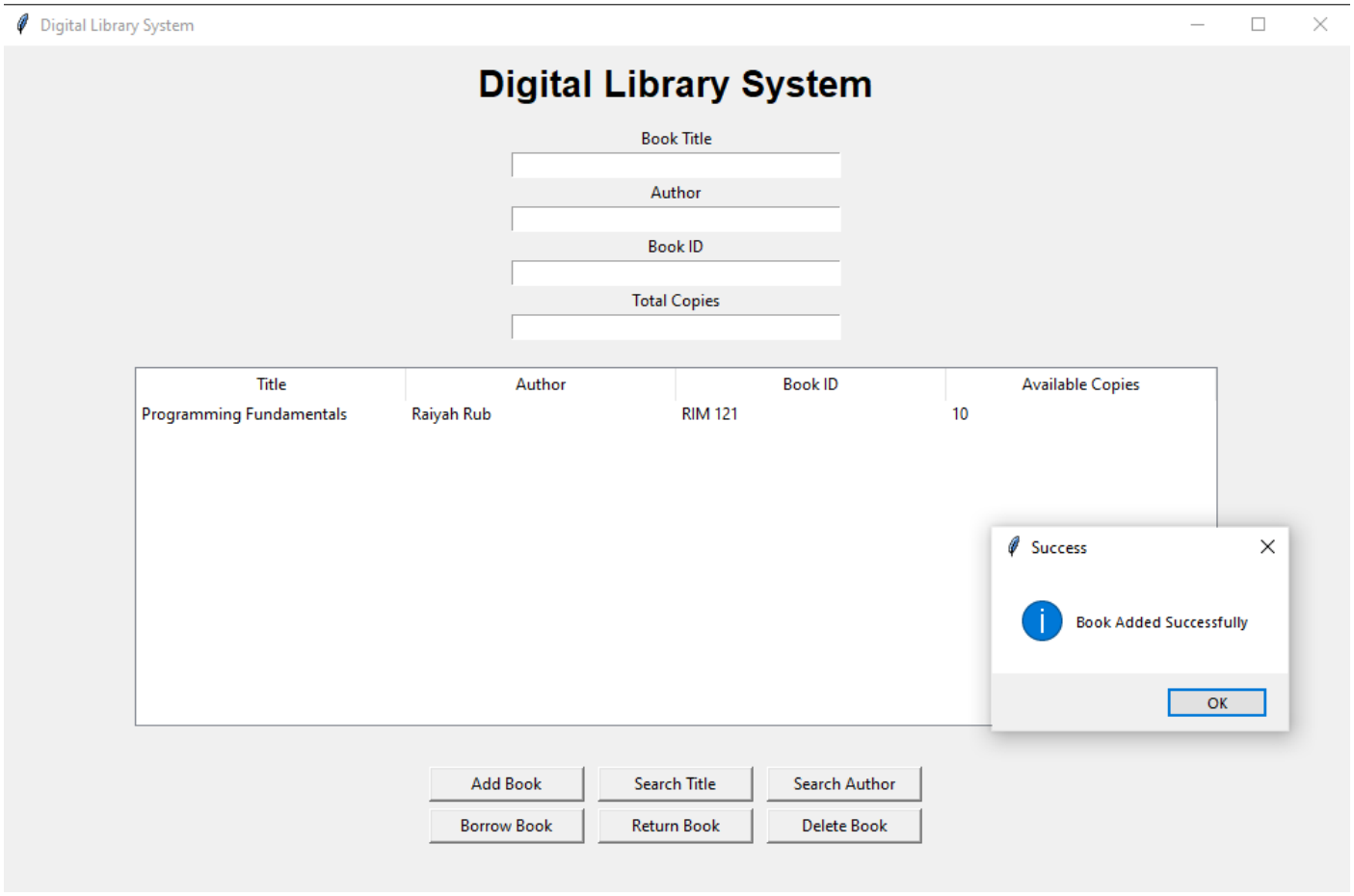
GUI Components:

- Labels
- Entry Fields
- Buttons
- Message Boxes
- Treeview Table

GUI Functions:

- refresh_table()
- clear_fields()
- add_book()
- search_title()
- search_author()
- borrow_book()
- return_book()
- delete_book()

Output Screens:



Main Interface:

The application displays:

- Book Title Field
- Author Field
- Book ID Field
- Total Copies Field

Buttons:

- Add Book
- Search Title
- Search Author
- Borrow Book
- Return Book
- Delete Book

A Tree view table is also used to display book records.

Add Book Output:

Book Added Successfully

Search Book Output:

Book Found

Title: Python Programming

Author: Ali Ahmed

Book ID: B101

Available Copies: 5

Borrow Book Output:

Book Borrowed Successfully

Return Book Output:

Book Returned Successfully

Delete Book Output:

Book Deleted Successfully

Observations:

1. The application successfully stores books in memory.
2. Search operations return accurate results.
3. Borrowing a book decreases available copies.
4. Returning a book increases available copies.
5. Deleted books are removed from the library records.
6. The GUI improves usability and interaction.
7. The system efficiently manages library operations.

Results and Discussion:

The Digital Library Management System was successfully implemented using Python and Tkinter.

All major functionalities including:

- Adding books
- Searching books
- Borrowing books
- Returning books
- Viewing books
- Deleting books

were executed successfully.

The use of Object-Oriented Programming improved code organization and maintainability. Tkinter GUI enhanced user interaction and made the application more user-friendly compared to a command-line interface.

Benefits:

The system provides several advantages:

- Easy book management
- Fast searching operations
- User-friendly interface
- Reduced manual effort
- Improved accuracy
- Better record organization
- Easy maintenance and future upgrades

Real World Applications:

The project can be used in:

Schools:

Managing school library books.

Colleges:

Tracking academic resources.

Universities:

Managing large collections of books.

Community Libraries:

Maintaining local library records.

Digital Learning Centers:

Providing book inventory management.

Book Rental Services:

Tracking borrowed and returned books.

Conclusion:

The Digital Library Management System is a successful implementation of Programming Fundamentals concepts using Python and Tkinter.

The project demonstrates the practical use of Object-Oriented Programming, data management, loops, conditions, functions, lists, and graphical user interfaces. The system efficiently performs all basic library operations including adding, searching, borrowing, returning, viewing, and deleting books.

Future enhancements may include database integration, login authentication, file storage, cloud synchronization, and advanced reporting features to make the system suitable for large-scale real-world applications.